QALA

Solution Factory Operating System

Extended Software Design Document v2.0

Field	Value
Document Type	Extended Software Design Document (SDD)
Version	2.0
Status	Draft
Scope	Full Platform — All Planes, Services & Subsystems
Audience	Engineering, Architecture, Product, Security, QA
Classification	Confidential

1 Executive Summary

Qala is an end-to-end Solution Factory Operating System — a polyglot, microservices-based platform governing the complete lifecycle of any solution type from inception to production and beyond.

This extended design document covers the full platform in depth, including: hermetic and reproducible build environments; integrated CI/CD/CM pipelines; build attestation and artifact signing; bug and defect tracking; version, change, and configuration control; SDE governance and lifecycle management; comprehensive testing, test analytics, and prototyping capabilities; performance and quality benchmarking; security, privacy, and threat management; tooling management; AI-driven intelligence across all subsystems; and backup, archiving, and disaster recovery.

> *Qala is solution-agnostic. It does not prescribe language, framework, or methodology. It defines the operational, structural, and governance conditions under which any solution — Application, System, Good, Product, Service, or Platform — can be produced consistently and at scale.*

2 Solution Structure Model

Every artefact produced inside Qala conforms to a strict seven-level hierarchy guaranteeing traceability, composability, and governance.

2.1 Containment Hierarchy

Level	Element	Contains
1	System	Applications
2	Application	Processes
3	Process	Components
4	Component	Interfaces
5	Interface	Messages (via Imports / Exports)
6	Message	Data Structures
7	Data Structure	Typed Data Fields

2.2 Solution Types

Type	Description
Application	A deployable software application delivered to end-users or integrated systems.
System	A coordinated collection of Applications serving a unified purpose.
Good	A tangible, physical, or digital deliverable produced by the factory.
Product	A commercially packaged, versioned, and distributable artefact.
Service	An ongoing capability delivered to consumers on a continuous basis.
Platform	A foundation on which other Solutions are built and operated.

2.3 Message Types

Message Type	Temporal Behaviour	Use Cases
Event (dynamic)	Discrete occurrence at a point in time; ephemeral; triggers reactive behaviour.	Build completed, threat detected, user logged in.
State (static)	Persistent condition or snapshot of data; describes current reality.	SDE configuration, artifact metadata, user profile.

2.4 Data Types

Category	Types
Scalar	bool, string, char, varchar
Numeric	int, float, double

Temporal	date
Composite	array, tuple, set, map, object
Reference	pointer
Custom	custom (user-defined extension)
Null	null (explicit absence of value)

3 Solution Development Environment (SDE)

An SDE is the foundational deployable unit of Qala: a fully self-contained, versioned, and governable workspace that defines all conditions for creating and operating solutions.

3.1 Core Properties

Property	Description
Deployable	Can be instantiated on a platform, cloud environment, or physical machine.
Configurable	All aspects — tooling, languages, variables, profiles — are parameterised.
Distributable	Can be packaged, replicated, and instantiated in new contexts.
Hermetic	Build environments within an SDE are fully isolated; all dependencies are frozen and immutable.
Versioned	Full version history of SDE configuration; any state can be inspected or restored.

3.2 SDE Composition

Element	Description
Environment Variables	Runtime configuration values injected at boot.
IDEs & Editor Configs	Integrated development environment configurations and plugins.
Languages & Runtimes	Programming languages, compilers, interpreters, and SDKs.
User Preferences	Per-user themes, workflow settings, and keyboard mappings.
Configuration	System-level and service-level configuration files (versioned).
Profiles	Named configurations for different deployment targets or roles.
Parameters & Options	Tuneable values and feature flags that modify SDE behaviour.
Toolkits / Toolsets / Tools	Hierarchical tooling organisation (see Section 10).
Solution Documentation	Charters, design docs, ADRs, reference files.
Content Management (CMS)	Solution-scoped versioned CMS for all content artefacts.
Backup & Recovery	Scheduled snapshots, incremental backups, and restore points.

3.3 SDE Lifecycle States

State	Description	Permitted Transitions
Provisioning	SDE being initialised; infrastructure allocated.	→ Active
Active	Running; available for all solution work.	→ Snapshotted, Suspended, Quarantined
Snapshotted	Point-in-time capture taken; SDE continues running.	→ Active, Rolled Back

Suspended	Paused; resources retained, not executing.	→ Active, Archived
Rolled Back	Restored to a prior snapshot state.	→ Active
Quarantined	Isolated by SEM due to confirmed threat; no ingress/egress.	→ Active (post-clearance)
Archived	Frozen and stored for audit; no active work.	→ Decommissioned (or Restored)
Decommissioned	Permanently shut down; resources released.	Terminal

3.4 SDE Governance

Every SDE is subject to the following governance controls:

- Ownership & RBAC: Each SDE has an owner and a defined set of role-based access controls. All access changes are audit-logged.
- Change Control: All configuration changes to an SDE are tracked in a change log with author, timestamp, reason, and before/after diff.
- Policy Enforcement: Platform-wide security, compliance, and coding-standard policies are continuously evaluated against all active SDEs.
- Lifecycle Approvals: Transitions to Archived or Decommissioned states require approval from a designated owner or administrator.
- Audit Trail: All lifecycle events (provisioning, snapshots, rollbacks, quarantine, decommission) are immutably recorded.

3.5 SDE Backup & Recovery

Backup System

- Supports full and incremental scheduled backups, as well as event-triggered backups (e.g., pre-deployment).
- Backup metadata captures: Backup ID, SDE ID, Developer ID, Environment Version / Snapshot, Timestamp, Backup Type (Full / Incremental), Storage Location (Local / Cloud), Status.
- Backups are stored in secure, versioned storage with configurable retention policies.

Recovery & Restore System

- Supports point-in-time restores, full environment rollbacks, and dependency rehydration.
- Developer-initiated restore via portal or IDE plugin.
- Restored SDEs are automatically validated for compatibility with active pipelines before resuming work.

Archival System

- Moves inactive or legacy SDEs, snapshots, and artefacts to long-term storage.
- Enforces retention policies for regulatory and audit compliance.
- Supports retrieval and reactivation of archived SDEs with full dependency restoration.

- Archive metadata captures: Archive ID, SDE ID / Snapshot ID, Linked Projects / Releases, Storage Location, Retention Policy, Retrieval Status.

4 Hermetic Build Environments

Qala enforces hermetic, reproducible build environments as a first-class platform capability. Every build is fully isolated, deterministic, and tamper-evident.

4.1 Principles

Principle	Description
Full Isolation	Build containers have no access to host network, filesystem, or ambient credentials. All inputs must be declared explicitly.
Immutability	Once a build environment is defined and locked, no dependency can change without a version bump and a new environment definition.
Environment-as-Code	All build environments are defined in version-controlled configuration files (Dockerfile, Nix expressions, or equivalent). The definition is the environment.
Frozen Dependencies	All dependencies — OS packages, language libraries, tools — are pinned to exact versions with cryptographic digests. No "latest" references are permitted.
Reproducibility	Given the same source commit and environment definition, any build must produce bit-for-bit identical outputs regardless of when or where it runs.

4.2 Build Environment Definition

- Environment definitions are stored in the SDE's version-controlled configuration repository under /build-envs/.
- Each definition specifies: base image digest, OS-level package manifest with pinned versions, language runtime version and digest, build tool versions, and environment variable declarations.
- Definitions are reviewed and approved through the Change Control system before activation (see Section 7).

4.3 Build Attestation & Signatures

Every build artefact produced in a hermetic environment is accompanied by a signed attestation record:

Attestation Field	Description
build_id	Unique identifier for this specific build invocation.
source_commit	Git commit SHA of the source code at build time.
env_definition_hash	SHA-256 digest of the build environment definition used.
builder_identity	Identity of the build agent / CI runner that executed the build.
inputs_hash	Merkle hash of all declared inputs to the build.
outputs_hash	SHA-256 digest of each produced artefact.
timestamp	UTC timestamp of build completion.
signature	Cryptographic signature over the attestation record, using the platform signing key.

provenance_standard	Attestation format (e.g., SLSA Provenance v1.0 compatible).

> *All artefacts must carry a valid, verifiable attestation before they can be promoted beyond the build stage. Attestation verification is enforced at every pipeline gate and at deployment time.*

4.4 SLSA Compliance Levels

SLSA Level	Requirements Met by Qala
Level 1	Build process is documented; provenance is generated.
Level 2	Build service is hosted; provenance is signed by the build service.
Level 3	Build environment is hermetic; all inputs are tracked; no persistent credential access during build.
Level 4	Two-party review of all build process changes; hermetic, reproducible builds enforced platform-wide.

5 CI / CD / CM — Continuous Integration, Delivery & Configuration Management

5.1 Continuous Integration (CI)

CI automatically builds, tests, and validates every code change, enforcing quality, security, and policy compliance before artefacts are produced.

CI Pipeline Stages

Stage	Activities
Source Trigger	Activated on commit, pull request, or scheduled event. Source commit is recorded in build attestation.
Environment Provisioning	Hermetic build environment is instantiated from the locked environment definition.
Dependency Fetch	All declared dependencies are fetched and their digests verified against the frozen manifest.
Static Analysis (SAST)	Code quality, complexity, linting, and security static analysis are run. Policy violations fail the build.
Build	Source is compiled / packaged within the hermetic environment. Build attestation is generated.
Unit Tests	All unit tests are executed. Coverage thresholds are enforced as pipeline gates.
Integration Tests	Service-level integration tests run against provisioned test environments.
Security Scan	Dependency vulnerability scan (SCA) and container image scan are executed.
Artefact Packaging	Build outputs are packaged, signed, and published to the Artefact Registry.
CI Result Event	BUILD_EVENTS published to Kafka with full build metadata and attestation reference.

5.2 Continuous Delivery / Deployment (CD)

CD automates the promotion and release of validated artefacts through staging, QA, and production environments.

Deployment Strategies

Strategy	Description	Use Case
Blue / Green	Two identical environments; traffic switches atomically to the new version. Instant rollback by reversing the switch.	Production releases requiring zero-downtime and fast rollback.
Canary	New version receives a small percentage of traffic, gradually increasing based on health metrics.	Risk-sensitive releases; gradual validation under real traffic.

Rolling	Instances are updated incrementally; old and new versions co-exist briefly.	Large fleets where full blue/green is cost-prohibitive.
Feature Flags	Code is deployed dark; features are enabled selectively via configuration.	Decoupling deployment from release; A/B testing.

CD Pipeline Gates

- All automated tests must pass (unit, integration, system, security).
- Build attestation must be verified before deployment.
- Performance benchmarks must not regress beyond configured thresholds.
- Manual approval gates are configurable for production deployments.
- Deployment events are published to BUILD_EVENTS and NOTIFICATIONS Kafka topics.

5.3 Configuration Management (CM)

CM governs the versioning, approval, and auditability of all configuration changes across SDEs, environments, and deployed solutions.

CM Scope

- SDE configuration (tooling, dependencies, environment variables, profiles).
- Build environment definitions.
- Service configuration (connection strings, feature flags, tuning parameters).
- Infrastructure configuration (Kubernetes manifests, Terraform state).
- Security policies and RBAC definitions.

CM Workflow

1. Change is proposed via a Configuration Change Request (CCR) with rationale and impact assessment.
2. CCR is reviewed and approved through the Change Control Board (CCB) workflow (see Section 7).
3. Approved change is applied and the new configuration version is committed to version control.
4. CM_EVENTS are published to Kafka; affected SDEs and pipelines receive notification.
5. Rollback to any prior configuration version is available via the SDE rollback mechanism.

6 Version Control Management

Qala provides first-class version control capabilities for all platform artefacts — not just source code.

6.1 Version-Controlled Artefact Types

Artefact Type	Version Control Mechanism
Source Code	Git repositories hosted within or integrated with the platform.
Build Environment Definitions	Stored in SDE config repo; changes require CCR approval.
Service Configuration	Versioned in platform config store; all changes tracked with author and timestamp.
Infrastructure Definitions	Terraform / Helm charts stored in VCS; changes go through CCB.
Security Policies	Versioned in the SEM policy store; updates require sign-off.
Test Plans & Suites	Versioned in the Test Management Service.
Solution Documentation	Versioned in the SDE CMS.
Artefacts & Binaries	Immutable, version-tagged entries in the Artefact Registry with full provenance.

6.2 Branching & Merge Strategy

- Platform enforces configurable branch policies: required reviewers, status checks, and signed commits.
- Feature branches are ephemeral; associated test and build environments are provisioned and torn down automatically.
- Merge to trunk triggers CI pipeline automatically.
- All merges are recorded with: author, timestamp, approvers, linked issues/bugs, and associated build attestation.

6.3 Tool Version Control

All tools within the Toolkit hierarchy (see Section 10) are version-controlled:

- Each Tool has a version manifest specifying its exact binary digest and provenance.
- Toolchain definitions reference specific tool versions; upgrading a tool version requires a new Toolchain definition and CCR.
- Tool version history is auditable and any prior version can be restored.

7 Change Control & Configuration Management Governance

All changes to platform configuration, build environments, security policies, and infrastructure are subject to a formal change control process.

7.1 Change Request Model

Field	Description
CCR ID	Unique change request identifier.
Title	Brief description of the change.
Requester	User or system initiating the change.
Change Type	Configuration / Infrastructure / Policy / Security / Tool / Process.
Affected Scope	SDEs, services, pipelines, environments, or solutions impacted.
Risk Assessment	Impact severity, probability of failure, rollback plan.
Justification	Business or technical rationale for the change.
Implementation Plan	Step-by-step execution plan with owner and timeline.
Rollback Plan	Steps to revert the change if it fails.
Approval Status	Pending / Approved / Rejected / Deferred.
Linked Events	Associated CM_EVENTS in Kafka for full auditability.

7.2 Change Control Workflow

6. Requester submits a CCR via the platform portal or API (POST /changes).
7. CCR is automatically risk-scored. High-risk changes require Change Control Board (CCB) approval.
8. Reviewers are assigned based on scope; they approve or reject with comments.
9. Approved changes are scheduled for implementation. A maintenance window is assigned if required.
10. Change is implemented; automated post-change validation runs (tests, scans).
11. CCR is closed with outcome recorded. All steps are published to CM_EVENTS.

7.3 Emergency Change Process

For critical production incidents requiring immediate action:

- Emergency changes bypass the standard CCB review but require post-implementation review within 24 hours.
- A simplified Emergency CCR is created with: incident reference, change description, implementing engineer, and approval from a designated emergency approver.
- All emergency changes are automatically flagged for next CCB review.

8 Bug Tracking & Defect Management

Qala provides an integrated defect tracking system linking bugs to releases, projects, environments, and CI/CD pipelines.

8.1 Bug Record Model

Field	Description
Bug ID	Unique identifier.
Title	Short description of the defect.
Severity	Critical / High / Medium / Low.
Priority	P1–P4 based on business impact.
Status	New / Triaged / In Progress / Fixed / Verified / Closed / Won't Fix.
Reporter	User or automated system that identified the defect.
Assignee	Developer or team responsible for resolution.
Environment	Environment in which the bug was observed.
Linked Release	Release version where the bug was introduced (if known).
Linked Feature / Epic	Associated product feature or development epic.
Steps to Reproduce	Detailed reproduction steps.
Linked Test Cases	Test cases added to prevent regression.
Resolution	Fix description and commit reference.
MTTR	Mean time to resolution (computed).

8.2 Defect Lifecycle

State	Trigger
New	Bug reported via portal, automated scan, or API.
Triaged	Product owner or tech lead assigns severity, priority, and owner.
In Progress	Developer begins investigation and fix.
Fixed	Fix committed; CI pipeline runs regression suite.
Verified	QA confirms fix in target environment.
Closed	Verified fix deployed to production.
Won't Fix	Deferred or accepted risk with documented rationale.

8.3 Integration Points

- CI/CD: Builds that introduce regressions (previously passing tests now failing) automatically create bug records linked to the offending commit.

- Security Scans: Vulnerabilities discovered during SAST/DAST/SCA generate security bug records in the defect tracker.
- Test Management: Failed test cases automatically link to open bug records.
- Release Management: No release can be promoted to production with open Critical or High severity bugs unless explicitly overridden with documented justification.

9 Release Management & Deployment Automation

Release Management governs the planning, approval, execution, and post-release monitoring of every solution deployment.

9.1 Release Record Model

Field	Description
Release ID	Unique identifier.
Release Name / Version	Semantic version (e.g., v3.4.1).
Release Type	Major / Minor / Patch / Hotfix / Rollback.
Target Environment	Staging / QA / Production.
Deployment Strategy	Blue-Green / Canary / Rolling / Feature Flag.
Release Owner	Engineer or team accountable for this release.
Linked Artefacts	Signed, attested artefacts included in this release.
Linked Change Requests	CCRs authorising configuration changes in this release.
Linked Bug Fixes	Defect records resolved by this release.
Test Gate Status	Pass/Fail status of all required test suites.
Benchmark Gate Status	Pass/Fail status of performance and quality benchmarks.
Security Gate Status	Pass/Fail status of security scans and policy checks.
Approval Status	Pending / Approved / Rejected.
Deployment Status	Scheduled / In Progress / Deployed / Rolled Back / Failed.

9.2 Release Gate Requirements

> *No release may proceed to production without passing all mandatory gates. Gate failures generate automated notifications and block the deployment pipeline.*

Gate	Requirement
Test Gate	All unit, integration, system, and security tests pass.
Coverage Gate	Code coverage meets or exceeds configured minimum thresholds.
Benchmark Gate	Performance and quality metrics do not regress beyond configured tolerances.
Security Gate	No open Critical or High severity vulnerabilities in code, dependencies, or container images.
Bug Gate	No open Critical or High severity unresolved bugs in the release scope.
Attestation Gate	All artefacts carry verified build attestations.
Change Control Gate	All configuration changes included in the release are covered by approved CCRs.

Approval Gate	Release owner and designated approvers have signed off.

9.3 Deployment Automation

- Deployment pipelines are fully automated from artefact promotion through environment-specific configuration injection to health-check validation post-deploy.
- Blue/Green deployments: traffic switch is automated but can be held at a configurable traffic-hold stage for manual verification.
- Canary deployments: traffic increment thresholds, observation windows, and automatic rollback triggers are all configurable per release.
- All deployment events are published to BUILD_EVENTS and NOTIFICATIONS Kafka topics in real time.

10 Testing, Test Management & Prototyping

Qala provides an integrated test lifecycle platform covering test planning, execution, defect linkage, environment provisioning, and AI-driven test intelligence.

10.1 Test Management Service

Core Capabilities

- Manages test cases, test plans, test suites, and test results with full version history.
- Links test cases to features, user stories, epics, bugs, and release records.
- Supports manual, automated, and AI-generated test cases.
- Tracks test coverage at unit, integration, system, and security levels.

Test Record Model

Field	Description
Test Case ID	Unique identifier.
Type	Unit / Integration / System / Acceptance / Security / Performance / Regression.
Status	Draft / Active / Deprecated.
Linked Feature	Associated feature or story.
Preconditions	Required state before test execution.
Steps	Ordered execution steps.
Expected Result	Defined expected outcome.
Automation Status	Manual / Automated / AI-Generated.
Last Execution Result	Pass / Fail / Skip / Error.
Linked Bugs	Defect records opened by failures of this test case.

10.2 Test Environments

- Test environments are provisioned automatically by the Environment Management subsystem on pipeline trigger.
- Environments are ephemeral by default: created for the test run and torn down on completion.
- Long-lived shared test environments (Staging, QA, Regression) are also supported with snapshot/restore capabilities.
- All test environments are defined as code, versioned, and provisioned from approved templates.

10.3 Defect Tracking Service

Integrates directly with the Bug Tracking system (Section 8). Additional test-specific capabilities:

- Automatic bug creation on test failure, linked to the failing test case, the build, and the environment.

- Flaky test detection: tests that non-deterministically pass/fail are flagged and routed to a flaky test queue.
- Regression analysis: compares test results across builds to identify newly introduced failures.

10.4 Prototyping Service

- Manages sandbox environments, Proofs of Concept (POCs), and interactive prototypes.
- Prototypes are isolated, time-bounded SDEs with configurable resource limits.
- Prototype outcomes (performance data, user feedback, defect discoveries) feed into the Product Management and Analytics systems.
- Prototypes can be promoted to full solutions or archived upon completion.

10.5 Test Analytics Service

- Aggregates test execution metrics across all projects, pipelines, and releases.
- Dashboard views: test pass rates, coverage trends, flakiness rates, average MTTR for test failures.
- Historical data enables trend analysis and predictive quality insights.

10.6 AI-Driven Test Intelligence

AI Capability	Description
Test Case Generation	Analyses code changes and existing test coverage to automatically generate new test cases targeting uncovered code paths.
Test Prioritisation	Ranks test suite execution order by risk, historical failure rate, and code change proximity to minimise feedback latency.
Predictive Defect Detection	Uses historical defect patterns to identify code areas with high defect probability before tests are written.
Flaky Test Remediation	Detects flaky tests, analyses root causes, and suggests or automatically applies fixes.
Self-Healing Tests	For dynamic UI or API environments, automatically updates test selectors and assertions when the target interface changes.
Prototype Feedback Analysis	Processes prototype telemetry and user feedback to generate structured insight reports.

11 Tooling Management

All tools used within the platform — from compilers to security scanners — are managed as first-class, versioned, governed assets.

11.1 Toolkit Hierarchy

Level	Entity	Description
1	Toolkit	A named, purpose-oriented collection of Toolsets (e.g., "Build Toolkit", "Security Toolkit", "Quality Toolkit").
2	Toolset	A grouped set of Tools addressing a specific concern within a Toolkit (e.g., "Static Analysis Toolset").
3	Tool	An individual executable capability (e.g., compiler, linter, scanner, formatter, test runner).
4	Toolchain	A linked, ordered sequence of Tools forming an automated workflow (e.g., Build Toolchain: fetch → compile → test → package → sign).

11.2 Tool Version Control

- Every Tool entry includes: name, version, binary digest (SHA-256), source/vendor, licence, and provenance attestation.
- Tool upgrades require a CCR (Section 7) before activation in any SDE or pipeline.
- Multiple tool versions can coexist in the registry; SDEs reference specific versions via Toolset definitions.
- Deprecated tool versions are archived but remain accessible for historical reproducibility.

11.3 Third-Party Vendor Tool Integration

- Vendor tools (IDEs, testing platforms, analytics tools, CI providers) are onboarded via the Vendor Integration Management subsystem.
- Onboarding workflow: vendor tool is registered, its API/plugin contract is validated, security scan is performed, and compatibility with target SDE profiles is verified.
- All vendor integrations are monitored for version updates, security advisories, and licence changes.
- Vendor tool configurations are version-controlled and subject to Change Control.

11.4 Tool Suite Management

- Platform administrators define pre-approved Tool Suites: curated sets of tools approved for specific project types, technology stacks, or compliance regimes.
- Developers select from approved Tool Suites when creating an SDE; deviations require Change Control approval.
- Tool Suite definitions are versioned and published to the platform toolkit registry.

12 Benchmarking & Performance Analysis

Qala provides integrated benchmarking across performance, quality, and process dimensions, enabling data-driven decisions and continuous improvement.

12.1 Performance Benchmarking

Metric Category	Metrics Captured
Compute	CPU utilisation (avg, p95, p99), memory usage, disk I/O, network throughput.
Application Performance	Response time (avg, p50, p95, p99), request throughput (RPS), error rate.
Scalability	Behaviour under load: stress test results, breaking point, recovery time.
Build Performance	Build duration, test execution time, pipeline end-to-end latency.

12.2 Quality Benchmarking

Metric Category	Metrics Captured
Code Quality	Cyclomatic complexity, maintainability index, code duplication percentage.
Standards Compliance	Linting violations, coding standard adherence rate.
Test Coverage	Unit, integration, and system test coverage percentages.
Defect Metrics	Defect density (bugs per KLOC), severity distribution, MTTR.
Technical Debt	Estimated remediation effort for known quality issues.

12.3 Process Benchmarking

Metric Category	Metrics Captured
Development Velocity	Story points completed per sprint, sprint goal achievement rate.
Deployment Frequency	Number of deployments per day / week per service.
Lead Time	Time from first commit to production deployment (DORA metric).
Change Failure Rate	Percentage of deployments resulting in a rollback or hotfix.
Release Efficiency	Release success rate, rollback frequency, hotfix frequency.

12.4 Cross-Application & Cross-Product Benchmarking

- Benchmark data is normalised across projects, applications, and products to enable apples-to-apples comparison.
- High-performing teams, processes, and frameworks are identified and surfaced as platform best-practice recommendations.
- AI-driven benchmarking identifies potential performance or defect hotspots before they manifest.

12.5 Benchmarking Dashboards & Alerts

Audience	Dashboard Focus
Executives	KPIs, delivery velocity trends, quality trajectory, security posture.
Architects	Code quality, technical debt, architecture fitness.
QA & Release Managers	Test coverage, defect density, performance stability across releases.
Engineering Teams	Sprint velocity, build performance, individual service benchmarks.

- Automated alerts fire when any benchmark metric falls below a configured threshold.
- Historical benchmark data is retained for trend analysis and predictive forecasting.

13 Security, Privacy & Threat Management

Security and privacy are platform-wide, continuous concerns — not bolt-on features. Every SDE, pipeline, and deployment is subject to active security governance.

13.1 Security Testing Integration

Test Type	Acronym	When Applied	Description
Static Application Security Testing	SAST	CI — every build	Analyses source code for vulnerabilities without execution. Enforced as a CI gate.
Software Composition Analysis	SCA	CI — every build	Scans all open-source and third-party dependencies for known CVEs.
Dynamic Application Security Testing	DAST	CD — staging/QA environments	Tests running applications by simulating external attacks.
Interactive Application Security Testing	IAST	CD — integrated test runs	Instruments the application during test execution to identify vulnerabilities in real-time.
Container Image Scanning	—	CI — image build	Scans container images for OS-level vulnerabilities and misconfigurations.
Infrastructure-as-Code Scanning	IaC Scan	CI — on IaC changes	Scans Terraform / Helm definitions for security misconfigurations before deployment.
Penetration Testing	PenTest	Scheduled / on-demand	Managed penetration testing with results tracked as security bug records.

13.2 Vulnerability Management

- Continuous vulnerability scanning is active across all SDEs, deployed services, and registered dependencies.
- All vulnerabilities are assigned a severity (Critical / High / Medium / Low / Informational) and a risk score.
- Critical vulnerabilities trigger immediate notifications and block all deployments from the affected SDE until remediated.
- Automated patching is supported for OS-level and dependency vulnerabilities, subject to Change Control approval.
- Remediation tracking links each vulnerability to an open bug record and a target fix version.

13.3 Threat Modelling & Risk Assessment

- Threat models are created for each solution, capturing: assets, attack surfaces, threat actors, threat scenarios, risk severity, probability, and mitigation plans.
- Threat models are version-controlled and reviewed at each major release.
- Risk records are linked to projects, applications, releases, and environments.
- The STRIDE and PASTA methodologies are supported as built-in threat modelling frameworks.

13.4 Security Event Management (SEM)

SEM Capability	Description
Threat Detection	Continuous analysis of SDE behaviour, network traffic, build outputs, and runtime signals for anomalies and known threat patterns.
Policy Enforcement	Platform security policies are defined centrally, versioned, and pushed to all SDEs. Policy violations generate SECURITY_EVENTS.
Secrets Vault	All credentials, API keys, and secrets are stored in the encrypted Secrets Vault. No secrets appear in code, logs, or configuration files.
SDE Quarantine	On confirmed threat detection, the SEM automatically isolates the affected SDE (no ingress/egress) pending investigation and clearance.
Cross-SDE Containment	Threats in one SDE trigger containment checks across all SDEs in the same Solution Factory.
Audit Trail	All security-relevant actions are immutably logged with actor, timestamp, and full event payload.

13.5 Privacy Management

- PII (Personally Identifiable Information) handling policies are enforced at the data layer: all PII fields are classified, and access is restricted to authorised roles.
- Data masking and anonymisation are applied automatically in non-production environments.
- Encryption at rest (AES-256) and in transit (TLS 1.3+) is enforced platform-wide.
- Data retention and deletion policies are configurable per data classification; automated enforcement runs on schedule.
- Audit logs capture all access to and modifications of sensitive data fields.

13.6 Compliance Reporting

Framework	Coverage
GDPR	Data inventory, consent management, right-to-erasure workflows, DPA audit logs.
CCPA	Consumer data access and deletion request tracking.
ISO 27001	Information security management controls, risk register, audit evidence.
SOC 2 Type II	Access control, availability, processing integrity, confidentiality, privacy controls.
SLSA	Build provenance, hermetic build evidence, artefact attestation records.

14 Project, Application, Product & Resource Management

Qala provides unified lifecycle management for all entities produced and operated within the platform.

14.1 Managed Entity Types

Entity	Management Scope
Project	Container for all work items, timelines, resources, and deliverables. Linked to one or more Solutions, SDEs, and Factories.
Application	Deployable software unit with full version history, release records, and environment associations.
Product	Commercially packaged solution with versioning, release notes, licencing, and distribution channels.
Platform	Foundation solution with API versioning, tenant management, and compatibility matrices.
Service	Operational capability with SLA tracking, consumer management, and versioned API contracts.
Good	Physical or digital deliverable with production tracking, distribution, and inventory management.
Resource	Any asset consumed by the factory: personnel, compute, licences, tools, or budget.

14.2 Project Management Capabilities

- Work item management: epics, stories, tasks, bugs, and sub-tasks with full hierarchy.
- Sprint and milestone planning with burndown and velocity tracking.
- Resource allocation: team assignments, workload visualisation, and skill-matrix matching.
- Dependency tracking across projects and applications.
- All project metrics feed into the Benchmarking & Analytics subsystem (Section 12).

14.3 Distribution, Release & Deployment

- Products and Services have configurable distribution channels (internal, partner, public).
- Release packages include: artefact manifest, signed attestations, release notes, migration guides, and rollback procedures.
- Distribution is gated: recipients must be in the authorised distribution list for the release tier.
- Deployment records link every deployed instance to its originating release, environment, and artefact set.

15 Environment Management & Workflow Automation

Environments are first-class, version-controlled, automatically provisioned assets. Every environment used in the platform is defined as code.

15.1 Environment Types

Type	Description
Developer SDE	Personal isolated environment; local or cloud-hosted; full IDE integration.
Feature Branch	Ephemeral environment provisioned per feature branch; torn down on merge.
Prototype / Sandbox	Time-bounded, resource-limited environment for POCs and experiments.
Test / QA	Shared environments for integration, system, and acceptance testing.
Staging	Production-mirror environment for final validation before release.
Production	Live environment; governed by strict change control and deployment gates.
Disaster Recovery	Standby environment for failover; kept in sync with production via replication.

15.2 Environment-as-Code

- All environments are defined in version-controlled configuration files specifying: base image, OS packages, middleware, databases, network topology, security hardening, and monitoring configuration.
- Environment definitions are stored in the SDE configuration repository and subject to Change Control approval.
- Drift detection: the platform continuously monitors running environments for configuration drift from their definitions and alerts on deviation.

15.3 Automated Provisioning

- Environments are provisioned via containerisation (Docker) and orchestration (Kubernetes / Helm charts) or cloud-native IaC (Terraform / Pulumi).
- Provisioning is triggered automatically by: pipeline events, pull request creation, prototype requests, or manual API calls.
- Environment cloning enables production-mirror staging environments to be provisioned on demand.
- Ephemeral environments have configurable TTLs and are automatically torn down on expiry or pipeline completion.

15.4 Workflow Automation

- Pre-configured project templates include full automated pipeline definitions: build, test, review, merge, and deploy workflows.

- Code review and approval workflows are automated: reviewers are assigned by ownership rules, reminders fire on SLA breach.
- Feature branch workflows: environment provisioning, pipeline execution, and tear-down are all automated.
- Notification and alert workflows are event-driven via Kafka, with configurable delivery channels.
- IDE plugins allow developers to trigger workflow actions (snapshot, deploy, scan) directly from their development environment.

16 Artefact, Package & Binary Management

A centralised, immutable registry for all compiled outputs, packages, containers, and third-party dependencies — with full traceability and provenance.

16.1 Artefact Registry

Field	Description
Artefact ID	Unique identifier.
Name & Version	Semantic version identifier.
Type	Binary / Container Image / Library / Package / Configuration / Documentation.
Digest	SHA-256 hash of the artefact content (immutable once stored).
Attestation Reference	Link to the signed build attestation record (Section 4.3).
Linked Build	Build ID that produced this artefact.
Linked Release	Release record(s) that include this artefact.
Linked SDE	SDE in which the artefact was produced.
Dependencies	Resolved dependency manifest with version and digest for each dependency.
Licences	SPDX licence identifiers for the artefact and all dependencies.
Retention Policy	How long the artefact is retained in the active registry before archival.
Status	Active / Deprecated / Archived / Recalled.

16.2 Dependency Management

- All dependencies are declared explicitly in lock files with exact version pinning and digest verification.
- Transitive dependencies are fully resolved and recorded in the dependency manifest.
- Dependency vulnerability scanning runs on every build and on a scheduled basis against the registry.
- Licence compliance checks flag any dependency with a licence incompatible with the project's declared licence policy.

16.3 Container Image Management

- All container images are stored in the platform registry with full layer provenance.
- Images are signed using the platform signing key and verified at deployment time.
- Base image updates trigger automated rebuild and retest pipelines for all dependent services.

16.4 Third-Party Software & Vendor Integrations

- Vendor tools and third-party packages are onboarded through the Vendor Integration Management subsystem.

- All vendor software is scanned for vulnerabilities, licence compliance, and supply-chain provenance before activation.
- Vendor integration configurations are version-controlled and subject to Change Control.
- Compatibility matrices track which vendor tool versions are approved for which SDE profiles and technology stacks.

17 System Architecture

Qala is implemented as a polyglot microservices platform. Services are organised into five planes, each implemented in the language best suited to its operational characteristics.

17.1 Architectural Planes

Plane	Language	Services
Kernel System	Rust	Service Registry, Command Bus, Event Aggregator, Health Monitor.
Control Plane	Go	API Gateway, User Identity, SDE Management, Workspace/CMS, Artefact/Package, Change Control, Bug Tracking, Release Management.
Execution Plane	Scala	CI/CD Workflow, Hermetic Build, Data Platform, Notifications, Benchmarking, Test Management.
Intelligence & Security Plane	Rust	AI Agents, SEM, Vulnerability Management, Privacy Management.
Platform Operations Plane	Go / Terraform	Environment Management, Tool Registry, Vendor Integration, Backup & Recovery, Audit & Compliance.

17.2 Service Registry

Service	Plane	Primary Responsibility
API Gateway	Control	Ingress, routing, authentication, rate limiting.
User Identity	Control	Users, RBAC, MFA, SSO, audit logging.
SDE Management	Control	SDE lifecycle, versioning, snapshot, rollback, governance.
Workspace / CMS	Control	File storage, content versioning, workspace management.
Artefact / Package	Control	Artefact registry, dependency management, signing.
Change Control	Control	CCR lifecycle, CCB workflows, CM audit trail.
Bug Tracking	Control	Defect records, triage, resolution, regression linking.
Release Management	Control	Release records, gate evaluation, deployment orchestration.
CI/CD Workflow	Execution	Pipeline definitions, trigger, execution, status.
Hermetic Build	Execution	Isolated build environments, attestation generation.
Test Management	Execution	Test cases, plans, suites, results, analytics.
Benchmarking	Execution	Performance, quality, and process metrics collection and analysis.
Data Platform	Execution	Analytics pipelines, warehouse, master data management.
Notifications	Execution	Multi-channel event-driven notification dispatch.
AI Agents	Intelligence	Recommendations, predictions, test generation, anomaly detection.

SEM	Intelligence	Threat detection, policy enforcement, quarantine, secrets vault.
Environment Management	Operations	Provisioning, cloning, drift detection, teardown.
Tool Registry	Operations	Tool versioning, toolchain definitions, vendor integrations.
Backup & Recovery	Operations	SDE backups, restore workflows, archival management.
Audit & Compliance	Operations	Immutable audit log aggregation, compliance reporting.

17.3 Event Topics (Kafka)

Topic	Key Publishers	Key Consumers
USER_EVENTS	Identity	SDE Mgmt, Workspace, Notifications, Audit
SDE_EVENTS	SDE Mgmt	AI, SEM, Notifications, Kernel, Backup
CM_EVENTS	Change Control	SDE Mgmt, CI/CD, Notifications, Audit
BUILD_EVENTS	CI/CD, Hermetic Build	Artefact, Notifications, Benchmarking, AI
ARTIFACT_EVENTS	Artefact/Package	Data Platform, Notifications, Audit
TEST_EVENTS	Test Management	Bug Tracking, Release Mgmt, Benchmarking, AI
RELEASE_EVENTS	Release Management	Notifications, Audit, Benchmarking, AI
DATA_EVENTS	Data Platform	Analytics, Notifications, SEM
SECURITY_EVENTS	SEM, Vulnerability Mgmt	Notifications, SDE Mgmt, Kernel, Audit
BENCHMARK_EVENTS	Benchmarking	Analytics, AI, Notifications
NOTIFICATIONS	All services	Notification Dispatch
AI_RECOMMENDATIONS	AI Agents	SDE Mgmt, Test Mgmt, Release Mgmt, Portal

18 API Reference

All external interaction with Qala is mediated through the API Gateway. The following table documents the full API surface.

18.1 Authentication & Identity

Method	Endpoint	Description
POST	/auth/login	Authenticate user; returns JWT.
POST	/auth/logout	Invalidate session token.
POST	/users	Create user account.
GET	/users/{id}	Retrieve user profile.
PUT	/users/{id}	Update user profile.
DELETE	/users/{id}	Deactivate user.

18.2 SDE Management

Method	Endpoint	Description
POST	/sdes	Provision a new SDE.
GET	/sdes	List all SDEs (filtered by owner, status).
GET	/sdes/{id}	Retrieve SDE details.
PATCH	/sdes/{id}	Update SDE configuration.
DELETE	/sdes/{id}	Decommission an SDE.
POST	/sdes/{id}/snapshot	Take a snapshot.
POST	/sdes/{id}/rollback	Rollback to a prior snapshot.
POST	/sdes/{id}/clone	Clone an SDE to a new instance.
POST	/sdes/{id}/backup	Trigger an on-demand backup.
POST	/sdes/{id}/restore	Restore from a specified backup.
POST	/sdes/{id}/solutions	Associate a solution.
DELETE	/sdes/{id}/solutions/{sid}	Remove a solution association.

18.3 Solutions, Projects & Products

Method	Endpoint	Description
POST	/solutions	Create a solution.
GET	/solutions	List solutions.
GET	/solutions/{id}	Retrieve solution details.
PATCH	/solutions/{id}	Update solution metadata.
DELETE	/solutions/{id}	Archive or delete a solution.
POST	/projects	Create a project.
GET	/projects/{id}	Retrieve project details.
POST	/projects/{id}/assign	Assign team or user to project.
PUT	/projects/{id}/status	Update project status.
POST	/factories	Create a Solution Factory.
POST	/factories/{id}/sdes	Add SDE to Factory.
DELETE	/factories/{id}/sdes/{sdeId}	Remove SDE from Factory.

18.4 CI/CD & Build

Method	Endpoint	Description
POST	/ci/pipelines	Create a pipeline definition.
GET	/ci/pipelines	List pipelines.
GET	/ci/pipelines/{id}	Retrieve pipeline details.
POST	/ci/pipelines/{id}/run	Trigger pipeline execution.
GET	/ci/pipelines/{id}/status	Query execution status.
GET	/ci/pipelines/{id}/logs	Retrieve execution logs.
GET	/builds/{id}/attestation	Retrieve build attestation.

18.5 Release & Change Management

Method	Endpoint	Description
POST	/releases	Create a release record.
GET	/releases/{id}	Retrieve release details.
POST	/releases/{id}/approve	Approve a release.
POST	/releases/{id}/deploy	Trigger deployment.

POST	/releases/{id}/rollback	Initiate rollback.
POST	/changes	Submit a Configuration Change Request.
GET	/changes/{id}	Retrieve CCR details.
POST	/changes/{id}/approve	Approve or reject a CCR.

18.6 Bugs & Testing

Method	Endpoint	Description
POST	/bugs	Report a bug.
GET	/bugs/{id}	Retrieve bug details.
PUT	/bugs/{id}/update	Update bug fields.
POST	/bugs/{id}/resolve	Mark bug resolved.
POST	/tests/cases	Create a test case.
GET	/tests/cases/{id}	Retrieve test case.
POST	/tests/plans	Create a test plan.
POST	/tests/runs	Execute a test run.
GET	/tests/runs/{id}/results	Retrieve test results.

18.7 Security, Artefacts & Analytics

Method	Endpoint	Description
GET	/security/threats	List active threats.
POST	/security/policy/update	Push security policy update.
POST	/security/scan_sde	Trigger SDE security scan.
GET	/security/secrets	List secrets (metadata only).
POST	/security/secrets	Store a secret in the vault.
GET	/security/logs	Query security audit events.
POST	/artifacts/upload	Upload an artefact.
GET	/artifacts/{id}	Retrieve artefact metadata.
GET	/artifacts/{id}/download	Download artefact binary.
DELETE	/artifacts/{id}	Retire an artefact.
GET	/recommendations	Retrieve AI recommendations.
POST	/analyze_sde	Request AI SDE analysis.
GET	/benchmarks	Query benchmark results.

GET	/data/analytics	Query analytics output.
POST	/analytics/predict	Submit predictive analytics request.

19 Delivery Roadmap

Year 1 — Foundation

- Q1: Core API Gateway, PostgreSQL schemas, User Identity, RBAC, MFA/SSO.
- Q2: SDE Management (local), Workspace/CMS, basic CI/CD, hermetic build environments (Phase 1).
- Q3: AI Phase 1 (recommendations, SDE optimisation), SEM Phase 1 (threat detection, secrets vault), IDE integrations.
- Q4: Artefact Registry, Bug Tracking, Change Control, developer onboarding portal.

Year 2 — Depth & Intelligence

- Q1: Test Management, Test Analytics, Defect Tracking integration, automated test provisioning.
- Q2: Release Management with full gate automation, Benchmarking Service, advanced CD (blue/green, canary).
- Q3: AI Phase 2 (predictive defect detection, test generation, self-healing tests), SEM Phase 2 (automated quarantine, DAST/IAST integration).
- Q4: Data pipelines, advanced analytics, multi-cloud SDE deployment, Privacy Management, GDPR/ISO27001 compliance reports.

Year 3 — Scale & Ecosystem

- Q1: Full multi-cloud SDE orchestration, Solution Factory marketplace (templates, plugins, extensions, vendor integrations).
- Q2: Scenario benchmarking, AI Phase 3 (cross-product insights, industry benchmark comparison).
- Q3: Global scaling, enterprise compliance add-ons, SaaS packaging and monetisation tiers.
- Q4: Ecosystem partner integrations, platform API stability guarantee (v2 public API).

KPIs

KPI	Target
Developer onboarding time	New SDE provisioned and first solution created < 30 minutes.
Hermetic build reproducibility	100% bit-for-bit reproducible builds for all attested artefacts.
CI pipeline success rate	> 95% across all active SDEs.
Snapshot / rollback latency	Snapshot < 60 seconds; rollback < 5 minutes.
SDE backup success rate	> 99.9% scheduled backup completion rate.
Threat detection to quarantine	< 90 seconds from threat signal to SDE quarantine.
Test case AI acceptance rate	> 50% of AI-generated test cases accepted by QA within 5 days.
Release gate automation coverage	100% of production deployments pass automated gate checks.
Multi-cloud provisioning time	< 10 minutes for any supported cloud target.
Vulnerability remediation SLA	Critical: < 24 hours; High: < 7 days; Medium: < 30 days.

20 Glossary

Term	Definition
SDE	Solution Development Environment. A deployable, configurable, hermetic workspace for solution creation and operation.
SF	Solution Factory. A networked, orchestrated collection of SDEs.
Hermetic Build	A fully isolated, reproducible build where all inputs are declared and frozen; no ambient environment access.
Build Attestation	A signed record proving a specific artefact was produced from a specific source commit in a specific hermetic environment.
SLSA	Supply-chain Levels for Software Artifacts. A security framework for build provenance and integrity.
CCR	Configuration Change Request. The formal record governing any change to platform configuration.
CCB	Change Control Board. The review and approval body for CCRs.
CM	Configuration Management. The discipline of versioning and governing all configuration artefacts.
CI	Continuous Integration. Automated build, test, and validation on every code change.
CD	Continuous Delivery / Deployment. Automated promotion and release of validated artefacts.
SAST	Static Application Security Testing. Analysis of source code for vulnerabilities without execution.
DAST	Dynamic Application Security Testing. Testing of running applications by simulating external attacks.
IAST	Interactive Application Security Testing. Instrumented testing during live test execution.
SCA	Software Composition Analysis. Scanning dependencies for known CVEs.
SEM	Security Event Manager. The Qala subsystem for threat detection, policy enforcement, and quarantine.
MTTR	Mean Time to Resolution / Recovery.
RBAC	Role-Based Access Control.
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege — threat modelling framework.
Toolchain	A linked sequence of Tools forming an automated workflow.
Toolkit	A named collection of Toolsets within an SDE.
Toolset	A grouped set of Tools addressing a specific concern within a Toolkit.
Artefact	A compiled, packaged, or binary output produced by a build process.
Snapshot	A point-in-time capture of an SDE state.
Quarantine	SDE isolation imposed by SEM on confirmed threat detection.
CMS	Content Management System — the versioned content layer within each SDE.

End of Document — Qala Solution Factory OS v2.0 | Extended SDD